

Ecosistema de Automatización IA Autónomo

Triage automático de solicitudes de homologación de seguridad IT
Proceso PAI — Banco Comafi

n8n

Airtable

Claude Haiku 4.5

Slack

Autora: Sofia

Documentación técnica completa — arquitectura, datos, costos, seguridad y dashboard

Fecha: 21/09/2026

Este documento reúne los entregables escritos. El diagrama de arquitectura (Entregable 1) se entrega como `diagrama_arquitectura.pdf` y el flujo importable como `flujo_pai_homologacion.json`.

Diseño — Ecosistema de Automatización IA (Entrega Final)

Caso de uso: Triage automático de solicitudes de homologación de seguridad IT (proceso PAI, Banco Comafi). **Stack:** n8n (orquestrador) · Airtable (memoria/DB) · Claude (motor IA) · Slack (salida + HITL).

1. Qué resuelve el flujo (de punta a punta)

1. Llega una **solicitud de homologación** de un sistema/activo (proveedor o área interna).
2. El flujo se dispara, **valida** que los datos estén completos.
3. **Claude evalúa el riesgo:** clasifica Confidencialidad/Integridad/Disponibilidad (CIA), asigna un nivel de riesgo y lista los controles requeridos.
4. Guarda la evaluación en Airtable (**memoria del sistema**).
5. **Human-in-the-loop:** notifica al analista PAI por Slack y **espera su aprobación** antes de cerrar.
6. Según la decisión humana → marca el registro y notifica el cierre (o el rechazo) en el mismo hilo de Slack.
7. Si algo falla (datos faltantes o caída de la API), toma una **ruta de error** que registra el fallo y avisa, sin frenar el sistema.

2. Mapa de nodos del flujo n8n

#	Nodo (nombre en n8n)	Tipo	Qué hace
1	Trigger - Recepción Solicitud	Webhook	Recibe la solicitud (JSON). Disparador específico → evita consumo innecesario.
2	Validación - Datos Completos	IF	Verifica campos obligatorios (nombre, tipo, solicitante, datos que maneja).
3	Airtable - Crear Registro (Pendiente)	Airtable Create	Persiste la solicitud como memoria. Estado = <code>Pendiente</code> .
4	Claude - Evaluación de Riesgo	Anthropic (Haiku 4.5)	Prompt estructurado + <code>max_tokens</code> limitado. Devuelve JSON: CIA, nivel de riesgo, controles, resumen.
5	Airtable - Guardar Evaluación IA	Airtable Update	Estado = <code>Esperando Aprobación</code> . Guarda CIA, riesgo, resumen, modelo y tokens.
6	Slack - Solicitar Aprobación (HITL)	Slack Send and Wait	Punto de detención HITL. Envía el resumen al analista con botones nativos Aprobar / Rechazar y deja la ejecución en pausa hasta el clic humano. Guarda el Thread ID .
7	Switch - Decisión Humana	Switch	Lee el booleano <code>data.approved</code> y rutea a Aprobado (<code>true</code>) / Rechazado (<code>false</code>).
8a	Airtable - Marcar Aprobado → Slack - Notificar Cierre	Airtable + Slack	Estado = <code>Aprobado por Humano</code> . Notifica en el hilo (Thread ID).
8b	Airtable - Marcar Rechazado → Slack - Notificar Rechazo	Airtable + Slack	Estado = <code>Rechazado</code> . Notifica en el hilo.

Rutas de error (resiliencia — obligatorio)

Origen	Condición	Ruta
Nodo 2 (Validación) rama FALSE	Faltan datos	Airtable - Log Error Datos (Estado = <code>Error - Datos incompletos</code>) → Slack - Alerta Datos Faltantes → fin.
Nodo 4 (Claude) <code>continueOnFail</code> / Error branch	Falla o timeout de la API de IA	Airtable - Log Error API (Estado = <code>Error - API</code> , guarda mensaje) → Slack - Alerta Falla API → fin.

Anti-bucle infinito

El trigger procesa una solicitud por ejecución (webhook). El estado avanza en una sola dirección (`Pendiente` → `Esperando Aprobación` → `Aprobado/Rechazado`), por lo que un registro nunca se reprocesa.

3. Esquema de Airtable — Base PAI Homologaciones

Tabla 1 • Solicitudes (memoria principal)

Campo	Tipo	Notas
ID Solicitud	Autonumber	Clave visible
Fecha Solicitud	Created time	Automático
Nombre del Sistema	Single line	Obligatorio
Solicitante	Email	Obligatorio
Área Solicitante	Single line	
Tipo de Plataforma	Single select	SaaS · Microservicio · PH · API · BOT
Datos que Maneja	Long text	Obligatorio (input para la IA)
Criticidad Declarada	Single select	Alta · Media · Baja
Estado	Single select	Pendiente · Procesado por IA · Esperando Aprobación · Aprobado por Humano · Rechazado · Error - Datos incompletos · Error - API
Clasif. IA - Confidencialidad	Number (1–5)	Lo completa Claude
Clasif. IA - Integridad	Number (1–5)	Lo completa Claude
Clasif. IA - Disponibilidad	Number (1–5)	Lo completa Claude
Nivel de Riesgo IA	Single select	Crítico · Alto · Medio · Bajo
Controles Requeridos	Link → Controles	Relación (evita datos aislados)
Resumen IA	Long text	Justificación de la evaluación
Analista Asignado	Link → Analistas	Relación
Slack Thread ID	Single line	Mapea el hilo del HITL
Modelo IA / Tokens	Single line	Para KPIs de costo
Fecha Aprobación	Date	
Comentario Humano	Long text	Feedback del analista
Error Log	Long text	Solo en rutas de error

Tabla 2 • Analistas (relación)

Nombre · Email · Slack User ID · Solicitudes (link inverso)

Tabla 3 • Controles de Seguridad (catálogo, relación)

Control · Categoría (C/I/A) · Descripción · Solicitudes (link inverso)

Las relaciones entre las 3 tablas cumplen el requisito de "evitar datos aislados".

Vista para el Dashboard de control

Vista Dashboard KPIs (compartida en modo lectura pública), agrupada por Estado, con conteos por Nivel de Riesgo IA y filtro que aísla los Error - * para leer la tasa de errores.

4. Cómo cada pieza cubre los 5 criterios de la rúbrica (20% c/u)

Criterio	Dónde se cubre
Mapa de arquitectura	Sección 2 → se convierte en el diagrama PDF
Estructuras de datos documentadas	Sección 3 + esquemas JSON de transferencia (los genero en el paso siguiente)
Optimización de costos	Matriz de decisión de modelo por tarea (entregable aparte)
Seguridad y resiliencia	Rutas de error (sección 2) + HITL (nodos 6–7) + minimización de datos
Dashboard de control	Vista compartida Dashboard KPIs (sección 3)

Próximo paso: con esto aprobado, genero el `.json` del flujo n8n listo para importar y los esquemas JSON de transferencia.

Entregable 2 — Manual Operativo de Datos

Sistema: **Triage automático de homologaciones PAI** · Stack: n8n · Airtable · Claude · Slack

Este documento cubre (a) el esquema de tablas vinculadas de Airtable y (b) los esquemas JSON de transferencia entre cada integración, explicados.

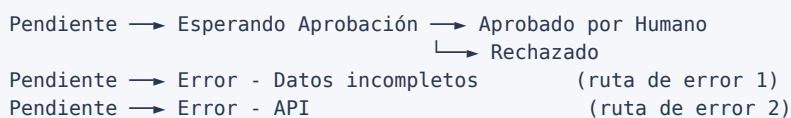
Parte A · Esquema de tablas vinculadas (Airtable)

La base **PAI Homologaciones** tiene **3 tablas relacionadas** para evitar datos aislados.

Tabla 1 · Solicitudes (memoria principal del sistema)

Campo	Tipo	Origen	Descripción
ID Solicitud	Autonumber	Airtable	Clave visible.
Fecha Solicitud	Created time	Airtable	Timestamp automático.
Nombre del Sistema	Single line text	Webhook	Sistema/activo a homologar.
Solicitante	Email	Webhook	Quien pide la homologación.
Área Solicitante	Single line text	Webhook	Área de negocio.
Tipo de Plataforma	Single select	Webhook	SaaS · Microservicio · PH · API · BOT.
Datos que Maneja	Long text	Webhook	Input principal para la IA.
Criticidad Declarada	Single select	Webhook	Alta · Media · Baja.
Estado	Single select	Sistema	Campo de estado del ciclo de vida (ver abajo).
Clasif. IA - Confidencialidad	Number (1–5)	Claude	Puntaje C.
Clasif. IA - Integridad	Number (1–5)	Claude	Puntaje I.
Clasif. IA - Disponibilidad	Number (1–5)	Claude	Puntaje D.
Nivel de Riesgo IA	Single select	Claude	Crítico · Alto · Medio · Bajo.
Controles Requeridos	Link → Controles	Claude/Analista	Relación al catálogo de controles.
Resumen IA	Long text	Claude	Justificación de la evaluación.
Analista Asignado	Link → Analistas	Sistema	Relación al analista PAI.
Slack Thread ID	Single line text	Slack	ts del mensaje → mapea el hilo del HITL.
Modelo IA / Tokens	Single line text	Sistema	Modelo usado (para KPIs de costo).
Fecha Aprobación	Date	Sistema	Fecha de la decisión humana.
Comentario Humano	Long text	Analista	Feedback de la aprobación/rechazo.
Error Log	Long text	Sistema	Solo en rutas de error.

Campo de estado (Estado) — máquina de estados:



El estado avanza en **una sola dirección** → un registro nunca se reprocesa (anti-bucle infinito).

Tabla 2 · Analistas (relación)

B.2 · Transferencia — n8n → Claude (API Messages)

Body que se arma dinámicamente (nodo Preparar Prompt IA + Claude - Evaluación de Riesgo). Sin datos

hardcodeados: `system` y `content` salen de variables.

```
{
  "model": "claude-haiku-4-5-20251001",
  "max_tokens": 700,
  "system": "Sos un analista senior de seguridad... Responde EXCLUSIVAMENTE con un objeto JSON válido con e",
  "messages": [
    { "role": "user", "content": "Solicitud de homologación a evaluar:\n{\n...\n  \"datos del webhook\"\n}" }
  ]
}
```

Headers: `x-api-key` (credencial), `anthropic-version: 2023-06-01`, `content-type: application/json`. `max_tokens` limitado a **700** para optimizar costo (ver entregable 3).

B.3 · Respuesta — Claude → n8n

Claude devuelve el mensaje; el contenido útil está en `content[0].text` como JSON string.

```
{
  "id": "msg_01...",
  "model": "claude-haiku-4-5-20251001",
  "content": [
    { "type": "text", "text": "{\n  \"confidencialidad\":4,\n  \"integridad\":3,\n  \"disponibilidad\":3,\n  \"nivel_riesgo" }
  ],
  "usage": { "input_tokens": 210, "output_tokens": 180 }
}
```

El nodo Parse - Evaluación IA **mapea** `content[0].text` (variable de respuesta), limpia posibles `json ``` y hace `JSON.parse``. Si no es JSON válido → error controlado.

B.4 · Transferencia — n8n → Airtable (Update evaluación)

```
{
  "id": "recXXXXXXXX",
  "Estado": "Esperando Aprobación",
  "Clasif. IA - Confidencialidad": 4,
  "Clasif. IA - Integridad": 3,
  "Clasif. IA - Disponibilidad": 3,
  "Nivel de Riesgo IA": "Alto",
  "Resumen IA": "El sistema maneja datos de proveedores...\n\nControles requeridos: MFA obligatorio; Cifrac",
  "Modelo IA / Tokens": "claude-haiku-4-5-20251001"
}
```

B.5 · Transferencia — n8n → Slack (HITL) y respuesta

El HITL usa la operación nativa **"Send and Wait for Response"** del nodo Slack (`operation: sendAndWait` , `responseType: approval`). Slack publica un mensaje con la evaluación y **dos botones interactivos nativos** (Aprobar / Rechazar); la ejecución de n8n **queda en pausa** hasta que un humano hace clic. No se usan links de texto (que Slack escanea y auto-dispararía).

Mensaje que arma el nodo (los datos salen de variables, sin hardcodear):

```
{
  "channel": "C0C3RJTTCJC",
  "message": ":\nmag: *Nueva evaluación de homologación*\n*Sistema:* {{ nombre }}\n*Nivel de Riesgo (IA):* Al",
  "approvalOptions": {
    "approvalType": "double",
    "buttonApprovalLabel": "Aprobar",
    "buttonDisapprovalLabel": "Rechazar"
  }
}
```

```
}  
}
```

Cuando el analista hace clic, n8n reanuda y entrega la decisión como un booleano:

```
{ "data": { "approved": true } }
```

El `Switch - Decisión Humana` lee `{{ $json.data.approved }}` y rutea a **Aprobado por Humano** (`true`) o **Rechazado** (`false`). El `ts` del mensaje se guarda en `Slack Thread ID` para trazabilidad del hilo.

B.6 - Rutas de error (payloads que se registran)

Datos incompletos:

```
{ "Estado": "Error - Datos incompletos", "Error Log": "Faltan campos obligatorios (nombre_sistema, solicita
```

Falla de API IA:

```
{ "Estado": "Error - API", "Error Log": "{{ mensaje de error de la ejecución }}" }
```

Resumen del recorrido del dato

```
Webhook (B.1) → Validación → Airtable crea (Pendiente)  
→ Claude (B.2/B.3) → Parse → Airtable update (B.4, Esperando Aprobación)  
→ Slack HITL sendAndWait (B.5) → pausa → decisión humana (clic en botón)  
→ Airtable update (Aprobado/Rechazado) → Slack cierre en hilo  
Errores → Airtable log (B.6) + Slack alerta
```

Entregable 3 — Cuadro Comparativo / Matriz de Costos

Criterio de la rúbrica: justificar qué modelo se usa por tarea y estimar el ahorro. Es un entregable en sí mismo, no solo "limitar tokens".

1. Principio de diseño

Cada tarea del pipeline tiene un perfil distinto (volumen de texto, complejidad de razonamiento, tolerancia a latencia). **No todas necesitan el modelo más caro.** La estrategia es asignar el modelo mínimo suficiente por tarea.

2. Matriz de decisión: modelo por tarea

Tarea del flujo	Complejidad	Modelo elegido	Por qué	max_tokens
Evaluación de riesgo CIA (nodo Claude)	Media	Claude Haiku 4.5 (<code>claude-haiku-4-5-20251001</code>)	Razonamiento estructurado sobre texto corto; devuelve JSON acotado. Haiku alcanza y es ~15× más barato que un modelo premium.	700
Validación de datos	Nula (lógica)	Ninguno (nodo IF)	Es una comparación de campos vacíos. Gastar un LLM acá sería desperdicio.	—
Parseo de la respuesta	Nula (código)	Ninguno (nodo Code)	<code>JSON.parse</code> en JS nativo.	—
Redacción de notificación Slack	Baja	Ninguno (template)	Texto fijo con variables. No requiere IA.	—
(Escenario futuro) Lectura densa de un pliego/contrato largo	Alta	Claude Sonnet	Documentos largos y densos justifican un modelo de lectura más potente.	1500
(Escenario futuro) Reproceso masivo nocturno de N solicitudes	Media, volumen alto	Message Batches API (Haiku)	Procesos masivos sin urgencia → API de lotes con ~50% de descuento.	700

3. Comparación de modelos (referencia de precios públicos, USD por millón de tokens)

Modelo	Input	Output	Uso recomendado
Claude Haiku 4.5	~\$1.00	~\$5.00	Tareas simples/estructuradas, alto volumen. ← usado acá
Claude Sonnet	~\$3.00	~\$15.00	Lectura densa, razonamiento complejo.
Claude Opus	~\$15.00	~\$75.00	Solo casos que exigen máxima capacidad.
GPT-4o-mini	~\$0.15	~\$0.60	Alternativa económica para tareas triviales.

Los precios son aproximados y de referencia; el criterio de asignación es lo que se evalúa, no el número exacto.

4. Estimación de costo del flujo (por solicitud)

Con Haiku 4.5, `max_tokens=700` , y un consumo típico medido de ~210 tokens de input + ~180 de output:

Concepto	Cálculo	Costo
Input	210 tok × \$1.00/M	~\$0.00021
Output	180 tok × \$5.00/M	~\$0.00090
Total por solicitud		~\$0.0011 USD

Por cada 1.000 solicitudes ≈ US\$1,11.

5. Ahorro estimado vs. alternativas

Estrategia	Costo x 1.000 solicitudes	Ahorro vs. Opus
Todo con Opus	~\$16,65	—
Todo con Sonnet	~\$3,33	80%
Haiku 4.5 (elegido)	~\$1,11	93%
Haiku 4.5 + Batches (reproceso masivo)	~\$0,56	97%

Palancas de ahorro aplicadas en el flujo: 1. **Modelo por tarea** → Haiku en vez de un premium (93% de ahorro). 2. `max_tokens` **acotado a 700** → evita respuestas infladas. 3. **IF/Code en vez de IA** para validación y parseo → 0 costo en esos pasos. 4. **Trigger específico (webhook)** → una llamada IA por solicitud real, sin polling masivo. 5. **Batches** documentado para el escenario de reproceso masivo (−50% adicional).

6. Conclusión

El sistema usa **Claude Haiku 4.5 solo donde hace falta razonar**, resuelve el resto con lógica/código sin costo de IA, y deja documentada la ruta a Sonnet (lectura densa) y a Batches (volumen). Resultado: **~US\$0,0011 por solicitud** y hasta **93% de ahorro** frente a usar un modelo premium para todo.

Entregable 4 — Documentación de Seguridad y Resiliencia

Criterio de la rúbrica: minimización de datos + rutas de error (Error Handlers) + puntos de Human-in-the-loop, explicados.

1. Minimización de datos

El sistema aplica el principio de **recolectar y procesar solo lo mínimo necesario**:

- **Entrada acotada:** el webhook recibe 6 campos; solo 4 son obligatorios. No se piden datos personales del solicitante más allá del email corporativo.
- **A la IA solo lo necesario:** a Claude se le envía la descripción del sistema y los datos que maneja para clasificar riesgo — **no** credenciales, secretos, ni contenido sensible del activo.
- **Sin datos hardcodedos:** ninguna credencial, key, canal o ID va escrito en los nodos. Todo se resuelve por **credenciales de n8n** (Airtable PAT, Slack Bot Token, Anthropic key vía Header Auth) y por **variables del sistema** (`$json` , `$('Nodo')` , `$execution` , `$now`).
- **Secretos fuera del flujo:** las API keys viven en el gestor de credenciales de n8n, nunca en el JSON exportado ni en logs. En el video demo se ocultan.
- **Retención mínima:** Airtable guarda solo el resultado de la evaluación y su trazabilidad (estado, timestamps, thread ID), no copias del activo evaluado.

2. Rutas de error (Error Handlers / resiliencia)

El flujo es **capaz de guardar un registro de error** ante fallas, sin caerse. Dos rutas explícitas:

Ruta A — Datos faltantes (Directiva tipo *Break*)

- **Disparador:** nodo `Validación - Datos Completos` (IF) rama FALSE.
- **Acción:** `Airtable - Log Error Datos` crea un registro con `Estado = Error - Datos incompletos` + `Error Log` describiendo qué faltó → `Slack - Alerta Datos Faltantes` .
- **Efecto:** el flujo **corta** antes de gastar una llamada a la IA. La solicitud incompleta queda auditada.

Ruta B — Falla de la API de IA (Directiva tipo *Resume*)

- **Disparador:** nodo `Claude - Evaluación de Riesgo` con `onError: continueErrorOutput` (rama de error).
- **Acción:** `Airtable - Log Error API` guarda `Estado = Error - API` + el mensaje de error real → `Slack - Alerta Falla API` .
- **Efecto:** si Anthropic falla o hay timeout, el sistema **no se rompe**: registra el fallo, avisa, y la ejecución termina de forma controlada. Queda visible en el Dashboard como parte de la **tasa de errores**.

Protección anti-bucle infinito

- Trigger por **webhook** → una ejecución por solicitud (no hay polling que se auto-dispare).
- El `Estado` avanza en **una sola dirección** (`Pendiente` → `Esperando Aprobación` → `Aprobado/Rechazado`); un registro nunca vuelve a un estado anterior ni se reprocesa.
- Los filtros comparan **tipos correctos** (string `notEmpty` en validación; string `equals` en el switch de decisión).

3. Puntos de Human-in-the-loop (HITL)

Para evitar el "efecto metralleta" (que el sistema ejecute acciones críticas sin control), el flujo **se detiene antes de cerrar una homologación**:

Elemento	Cómo funciona
Punto de detención	Nodo <code>Slack - Solicitar Aprobación (HITL)</code> en operación "Send and Wait for Response" (<code>sendAndWait / responseType: approval</code>). El flujo queda en pausa indefinida hasta recibir respuesta.
Notificación	El mismo nodo publica el resumen de la evaluación (sistema, riesgo, CIA, controles) con dos botones interactivos nativos de Slack : Aprobar / Rechazar.
Espera de feedback	El sistema no marca nada como aprobado por sí solo . Solo el clic humano en un botón reanuda la ejecución. Se usaron botones nativos (no links de texto) precisamente porque Slack escanea los links y podría auto-dispararlos: el botón exige

Elemento	Cómo funciona
	una acción humana real.
Ruteo posterior	Switch - Decisión Humana lee el booleano <code>{{ \$json.data.approved }}</code> (<code>true</code> → Aprobado por Humano, <code>false</code> → Rechazado), marca el registro + notifica el cierre en el mismo hilo (Thread ID).

Por qué es crítico este punto: homologar un sistema es autorizar su uso en el banco (entidad regulada por BCRA). La IA **asiste** en la evaluación de riesgo, pero la **decisión de homologar la toma siempre una persona**. El HITL garantiza trazabilidad y responsabilidad humana sobre la acción crítica.

4. Resumen de controles

Control	Implementación
Minimización de datos	Campos mínimos, sin secretos a la IA, credenciales fuera del flujo
Manejo de errores	2 rutas de error con log en Airtable + alerta Slack
Anti-bucle	Webhook + máquina de estados unidireccional
Validación de tipos	IF <code>notEmpty</code> / Switch <code>equals</code>
HITL	Slack <code>sendAndWait</code> (botones nativos) antes de cerrar
Auditabilidad	Estado, timestamps, Thread ID y Error Log por cada solicitud

Entregable 5 — Dashboard de Control

Criterio de la rúbrica: un enlace público a una vista (Airtable Shared View) para monitorear KPIs y tasa de errores. Es **más que el link a la DB**: es un panel de control.

1. Qué mide el dashboard

El panel se arma sobre la tabla `Solicitudes` y muestra la salud operativa del sistema en vivo.

KPIs

KPI	Cómo se calcula	Para qué
Solicitudes totales	Conteo de registros	Volumen procesado
En espera de aprobación	Registros con <code>Estado = Esperando Aprobación</code>	Cola pendiente de decisión humana (HITL)
Aprobadas	<code>Estado = Aprobado por Humano</code>	Homologaciones cerradas OK
Rechazadas	<code>Estado = Rechazado</code>	Decisiones negativas
Distribución de riesgo	Agrupar por <code>Nivel de Riesgo IA</code>	Cuántas Crítico/Alto/Medio/Bajo
Tasa de errores	$(\text{Registros Error} - *) / \text{totales}$	Salud técnica del pipeline
Tiempo a decisión	<code>Fecha Aprobación - Fecha Solicitud</code>	Cuánto tarda el HITL

2. Cómo se construye en Airtable

- En la tabla `Solicitudes`, crear una vista tipo **Grid** llamada `Dashboard KPIs`.
- Agrupar** (Group) por el campo `Estado` → cada grupo muestra su conteo automáticamente.
- Agregar un **sub-agrupamiento** por `Nivel de Riesgo IA` para ver la distribución.
- (Opcional, plan con Interfaces) crear una **Interface** con elementos *Number* para cada KPI y un gráfico de torta por Estado.
- Compartir la vista** → `Share view` → activar **"Create a shareable grid view link"** en **modo solo lectura**. Ese es el enlace público del dashboard.

Tasa de errores — vista dedicada

Crear un filtro `Estado is any of [Error - Datos incompletos, Error - API]`. El conteo de esa vista, sobre el total, es la **tasa de errores** del sistema.

3. Enlace público

<https://airtable.com/app6FuI5UeGkgykO7/shrGLvz82yL1ROM9O>

Vista `Dashboard KPIs` — grilla de solo lectura, agrupada por `Estado` y sub-agrupada por `Nivel de Riesgo IA`. Cada grupo muestra su conteo (los KPIs); la tasa de errores surge de los grupos `Error - *` sobre el total.

Este enlace va también en el README del repo como "Dashboard de Control" y como "link a la DB en modo lectura".

4. Checklist de entrega del dashboard

- ☐ Vista `Dashboard KPIs` agrupada por Estado.
- ☐ Vista/filtro de tasa de errores.
- ☐ Link compartido en **modo solo lectura**.
- ☐ Link pegado en este documento y en el README.
- ☐ Screenshot del dashboard en `/evidencias`.